nodes where the container is running, making the upgrade faster and cheaper (especially on cloud platforms, which bill for the bandwidth used).

Let's update the previous Dockerfile to containerize Catalog Service more efficiently using the layered-JAR mode. Using this new strategy means doing some preparation work to copy the JAR file into the image and expand it into the four layers described previously. We don't want to keep the original JAR file inside the image, or our optimization plan will not work. Docker provides a solution for that: *multi-stage builds*.

We'll divide the work into two stages. In the first stage we extract the layers from the JAR file. The second stage is where we place each JAR layer into a separate image layer. In the end, the result of the first stage is discarded (including the original JAR file), while the second stage will produce the final container image.

Listing 6.3 More efficient Dockerfile to build a Catalog Service image

```
FROM eclipse-temurin:17 AS builder                          ❶
WORKDIR workspace
ARG JAR_FILE=build/libs/*.jar                               ❷
COPY ${JAR_FILE} catalog-service.jar                        ❸
RUN java -Djarmode=layertools -jar
➥ catalog-service.jar extract                              ❹

FROM eclipse-temurin:17                                     ❺
WORKDIR workspace
COPY --from=builder workspace/dependencies/ ./              ❻
COPY --from=builder workspace/spring-boot-loader/ ./
COPY --from=builder workspace/snapshot-dependencies/ ./
COPY --from=builder workspace/application/ ./
ENTRYPOINT ["java",
➥ "org.springframework.boot.loader.JarLauncher"]           ❼
```

❶ OpenJDK base image for the first stage

❷ Builds argument specifying the location of the application JAR file in your project

❸ Copies the application JAR file from the local machine into the image inside the "workspace" folder